

Stage HU

You

July 2, 2026

Abstract

Here we present algorithmic aspects regarding the sampling of point sets that are well-distributed in space. Such point patterns are crucial for numerical applications, serving notably as a foundation for efficient Monte-Carlo methods. In this context, we present and compare different methods of sampling and diagnosing well-distributedness, grounded in the concept of hyperuniformity.

1 Hyperuniformity

Let us fix a finite domain Ω of $\mathbb{R}^D$. In theoretical models, Ω would be the entire infinite $\mathbb{R}^D$ space, but here we will focus on numerical implementations, where Ω is finite, typically either the unit hypersphere or the unit hypercube. Let V denote its volume. A distribution of N particles is sampled on it from a collective probability distribution. N itself might be a random variable with a finite expectation, such that we can denote ρ as the expectation of N/V .

Two major examples of such sampling are gases and crystals. Crucially, they show very distinct behavior in terms of well-distributedness.

Gases, modeled by the Poisson point process, consist of i.i.d. uniform particles. The Poisson point process is well-distributed on average, in the sense that over several repetitions of the sampling process, the density of points that falls in a spherical observation sub-window of volume v has expectation ρ . But if we measure the actual empirical density of the sub-window in a single realization, it will not be exactly ρ . Empirical density fluctuates around ρ , with a variance equal to ρv if N itself is the proper random Poisson variable (or $\rho v(1 - v/V)$ if we fix a common N in advance for all realizations). The variance notably scales as v when we make v bigger.

On the other hand, crystals—modeled by taking a periodic structure (e.g., a grid lattice) and then applying a random shift U to the whole crystal to make the distribution uniform, and restricting it to Ω —have remarkably reduced density fluctuations compared to the Poisson process. The variance this time scales slower than v , instead scaling as the surface s of the spherical sub-window.

It has been proven that surface scaling is the smallest possible regime of fluctuations. Interestingly, a wide variety of point processes are both disordered like gases (no periodic structure) but well-distributed like crystals (sub-volumic fluctuations). Crucially, as they are disordered, they are isotropic (no privileged direction, unlike crystals). Among them, one can cite Poisson disks and the Ginibre ensemble. A point process with sub-volumetric density fluctuations is said to be hyperuniform.

2 Diagnosing Hyperuniformity

Whereas density fluctuations scaling, which is the metric for hyperuniformity, are defined across independent repetitions of the whole sampling process (we sample new point patterns again and again), in practical applications it is often required to diagnose hyperuniformity from a single realization. Whereas it could be done with, e.g., shifted repetitions of the observation sub-window that would screen sufficiently distant parts of the domain, the most used tools are derived from a link between hyperuniformity and the two-point correlation function (which measures the infinitesimal probability to find a particle at a given offset from a reference particle).

The effective tool for hyperuniformity is the structure factor, which is the spectral representation of the two-point correlation obtained by taking an appropriate Fourier transform of it: Hyperuniformity is equivalent to the vanishing of the structure factor when the wave number $k \rightarrow 0$.

The strength of hyperuniformity can itself be measured by the decreasing rate of $S(k)$ whenever $S(k)$ follows a power decay. Common diagnoses rely on defining and measuring an empirical estimator of the structure factor. In the case where the domain Ω is the unit hypercube, the standard estimator is the scattering intensity, simply defined as $\frac{1}{N} \left| \sum e^{i\mathbf{w} \cdot \mathbf{x}} \right|^2$, where $\mathbf{w}$ is any wavevector of the spectral domain of the form $\frac{2\pi}{L}\mathbf{k}$, with $\mathbf{k} \in \mathbb{Z}^D \setminus \{0\}$.

3 Stealthy

Beyond hyperuniformity, some point processes are so strongly hyperuniform that the structure factor not only has a 0 limit when $k \rightarrow 0$, but is effectively 0 up to algorithmic precision on a full range of k 's. We will discuss this in detail through examples.

4 Sampling methods

We tested a variety of sampling methods, focusing on the ones for which an open-source Python implementation was easily available to us and able to scale fast to at least 100,000 points.

- Poisson Disk
- Sobol
- CCVT
- Void and Clusters
- GBN
- NUFFT
- STIT

We focused more specifically on GBN as a case study for common aspects of hyperuniformity and as the only one that shows good scaling with respect to dimension to our knowledge. We put significant effort into extending it with a fast, scalable algorithm (namely RGBN).

5 RGBN

5.1 Toroidal hypercube

Boundaries are an issue for many sampling algorithms and for hyperuniformity diagnoses. When the domain Ω is the unit hypercube, a common technique is to periodize it, using toroidal boundary conditions. In what follows, accounting for a particle requires accounting for all periodized versions of it. A particle at position $(x, y, z, \dots)$ automatically has numerical twins at positions e.g., $(x+1, y, z, \dots)$.

5.2 GBN

GBN (Ahmed et al.) stands for Gaussian Blue Noise. It relies on a collective coordinate descent: particles interact through a short-ranged energy field. In the case of Gaussian blue noise, the energy is a Gaussian, and it is minimized by a gradient descent.

GBN implementation is quite straightforward once one realizes that the loop over all twins of a point can be factorized thanks to the special nature of the Gaussian kernel, and once the Gaussian bandwidth and learning rate are fixed. We note that, as an alternative to the factorization trick on the twins (which would not work for other kernels), we propose replacing the spatial kernel $f(u-v)$ by $f(\sin(2\pi(u-v)))$. This is what we used to model toroidal boundaries instead of using twins, because we found it slightly simpler and more generalizable for non-factorizable kernels.

Complexity

The complexity of each gradient step is $\mathcal{O}(N^2)$ (loop over all the particle pairs). The number of required steps itself is $\mathcal{O}(N)$ (at least we have strong empirical evidence for that). Interestingly, the $\mathcal{O}(N)$ scaling of the number of steps in iterative methods based on local interaction seems quite common (see, e.g., “Blue-Noise Point Sampling using Kernel Density Model” by Fattal). This makes the total complexity $\mathcal{O}(N^3)$. Note that this $\mathcal{O}(N^3)$ should not be as frightening as other $\mathcal{O}(N^3)$ algorithms, as the $\mathcal{O}$ constant is very small, especially on GPU for numerical reasons: it involves only a few lines of code, internally using native vectorization, which is very machine-friendly.

Why loop over all the particles? The Gaussian kernel is local by nature. Most interactions are negligible. But from an implementation perspective, computers suffer from major slowdowns when asked to perform random memory access instead of linear streaming of plain tensors. This is a common issue in sparse matrix computations, where in practice, accounting for the sparsity of a matrix gives speed benefits compared to using its plain representation only when the sparsity ratio is below 1%.

5.3 Boosts: RGBN

RGBN scales GBN to a fully linear complexity in N instead of $\mathcal{O}(N^3)$, at the cost of an effectively negligible quality decrease in hyperuniformity and increased coding effort.

1. Sparsity boost

We just said that sparsity induced by kernel truncation is not pertinent if it leads to unstructured memory access. But we built a novel data structure able to benefit both from sparsity and from structured tensor streaming, called the “sqaurennet”. The sqaurennet simply stores the points themselves as a tensor by optimally permuting the initial indices of the points to make points that are close in space land close in the tensor. With truncation to the 100 nearest neighbors, the Gaussian kernel is both still exact up to machine precision and now $\mathcal{O}(N)$ per gradient step, which makes the total complexity $\mathcal{O}(N^2)$ instead of $\mathcal{O}(N^3)$.

2. Spectral boost

A simple spectral observation immediately reduces the complexity from $\mathcal{O}(N^2)$ to something much smaller (maybe $\mathcal{O}(N\sqrt{N})$): most of the gradient descent iterations are wasted on optimizing the smallest frequencies, which are the most difficult to capture with a spatial kernel. But there are very few small wavevectors. Using a hybrid kernel—with the spatial GBN loss and an extra spectral loss on the 100 first hard-to-reach wavevectors—converges faster than the spatial kernel alone.

3. Recursivity boost

The solution for reaching a fully linear complexity term comes from Fattal again: as observed, recursivity allows controlling all spatial scales with the same strength, and thus optimizing with the same speed instead of having disfavored frequencies.